

UNIDAD 2.2

Introducción a React

2ª parte

Props, estado y eventos: convertimos el componente Alert en un componente configurable e interactivo

Props

useState

Eventos

React DevTools

AGENDA

Lo que veremos en esta parte

Ya lo vimos en la UT 02.1

01 Ventajas de React

02 Configuración de un proyecto

03 Estructura de la aplicación

04 Creación de un componente

En esta parte

05 Uso de props

06 Uso del estado

07 Uso de eventos

08 Herramientas de desarrollo

PUNTO DE PARTIDA

Nuestro Alert todavía es rígido

El componente Alert de la UT 02.1 siempre muestra lo mismo, no se puede cerrar y no avisa a nadie de lo que ocurre.



Contenido fijo

El título y el mensaje están escritos dentro del propio componente.

Solución

Props



No se puede cerrar

La alerta permanece en pantalla pase lo que pase.

Solución

Estado



No avisa de nada

El componente padre no sabe cuándo el usuario interactúa con la alerta.

Solución

Eventos

PROPS

Props: datos del padre al hijo



```
// DatosContacto.jsx
export function DatosContacto(props) {
  console.log(props.nombre);
  console.log(props.email);
}
```

```
// App.jsx
<DatosContacto nombre="Fred" email="fred@ejemplo.com" />
```

props es un objeto con las propiedades que elijamos. Se pasan como **atributos JSX** y sus nombres deben coincidir con los que espera el componente.

PROPS EN ALERT

Definimos las props de Alert

```
export function Alert(props) {  
  return (  
    <div>  
      <div>  
        <span role="img"  
          aria-label={props.type === 'warning'  
            ? 'Advertencia' : 'Información'}>  
          {props.type === 'warning' ? '⚠️' : 'ℹ️'}  
        </span>  
        <span>{props.heading}</span>  
      </div>  
      <div>{props.children}</div>  
    </div>  
  );  
}
```

type

'information' o 'warning'. Decide el icono y su etiqueta accesible.

heading

El texto del título de la alerta.

children

Prop especial: el contenido escrito entre las etiquetas del componente.

PROPS EN ALERT

Pasamos las props desde App

```
// App.jsx
<Alert type="information" heading="Success">
  ¡Todo va muy bien!
</Alert>
```



```
// props que recibe Alert
{
  type: 'information',
  heading: 'Success',
  children: '¡Todo va muy bien!'
}
```

Resultado en el navegador



Success

¡Todo va muy bien!

Desestructurar y dar valores por defecto

Antes

```
export function Alert(props)
...
<<span>{props.heading}</span>
  <div>{props.children}</div> </div>
```

Usamos las propiedades vía **props**.

Después

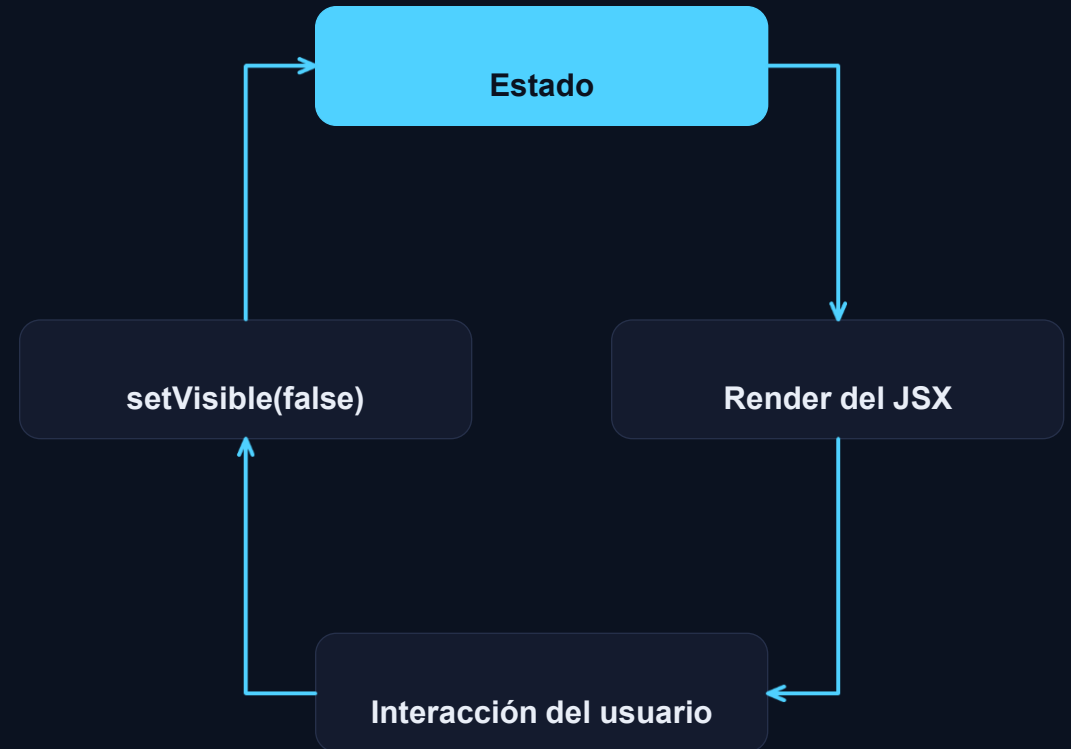
```
export function Alert({ type = 'information', heading, children })
...
<span>{heading}</span>
<div>{children}</div> </div>
```

Desestructuramos y **type = 'information'** es el valor por defecto si no se pasa la prop.

ESTADO

El estado: datos que cambian

- ☑ Es una variable especial que puede cambiar a lo largo de la vida del componente.
- ⚡ Cuando cambia, React vuelve a renderizar el componente: un re-render.
- ⚙ Cada componente define el estado que necesita. Algunos no necesitan ninguno.



ESTADO

El hook useState

```
const [visible, setVisible] = useState(true);
```

Valor actual

visible contiene el valor del estado en cada render.

Función de actualización

setVisible cambia el valor y provoca un re-render. Convención: set + nombre.

Valor inicial

true. Si no se pasa ningún valor, el estado empieza como undefined.

Varios estados

Se llama a useState una vez por cada dato.
Se importa con **import { useState } from 'react'**.

```
const [cargando, setCargando] = useState(true);  
const [error, setError] = useState();
```

ESTADO

Un estado visible para Alert

0 Importa el hook

1 Desde el módulo 'react' con una importación ES6 con nombre.

02 Declara el estado

visible empieza en true: la alerta se muestra.

03 Devuelve null

Si visible es false, el componente no renderiza nada.

```
import { useState } from 'react';

export function Alert({ ... }) {
  const [visible, setVisible] = useState(true);
  if (!visible) {
    return null;
  }
  return (
    // ... JSX de la alerta
  );
}
```

ESTADO

Botón de cierre condicional con &&

```
// Alert.jsx
{closable && (
  <button aria-label="Cerrar">
    <span role="img" aria-label="Cerrar">✖</span>
  </button>
)}
```

```
// App.jsx
<Alert type="information" heading="Success" closable>
  ¡Todo va muy bien!
</Alert>
```

Un atributo booleano sin valor vale **true**: closable equivale a closable={true}.

closable es un valor **verdadero** → se muestra el botón



Success

¡Todo va muy bien!



closable es un valor **falso** → la expresión se corta



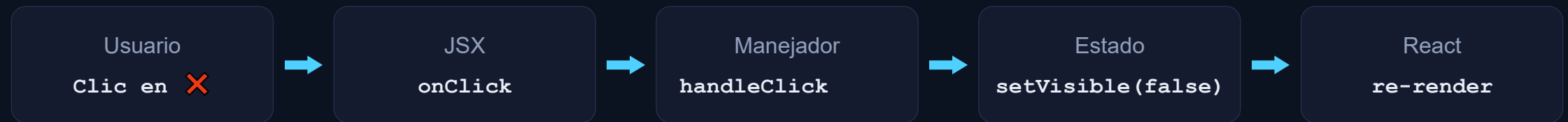
Success

¡Todo va muy bien!

EVENTOS

Eventos: reaccionar a la interacción

Un manejador de eventos es una función que se ejecuta cuando ocurre un evento sobre un elemento.



Los eventos de React son una capa que **envuelve los eventos nativos** del navegador.



Se registran en JSX con atributos en camelCase:

```
<button onClick={handleClick}>
```

EVENTOS

Implementamos el manejador de clic

```
// Dentro de Alert, antes del return
function handleCloseClick() {
  setVisible(false);
}

<button aria-label="Cerrar" onClick={handleCloseClick}>
```

Función declarada

```
function handleCloseClick() {
  // ...
}
```

Función flecha

```
const handleCloseClick = () => {
  // ... };
```

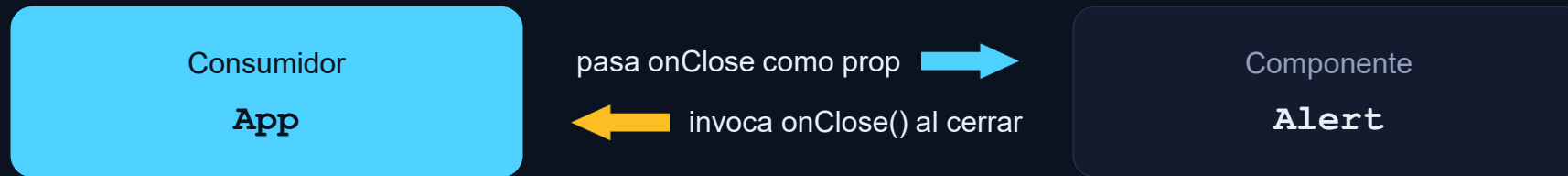
En línea

```
<button onClick={() => {
  // ... }}>
```

El manejador se define **dentro de Alert** para tener acceso a las props y al estado.

EVENTOS

Creamos un evento propio: onClose



```
// Alert.jsx
function handleCloseClick() {
  setVisible(false);
  if (onClose) {
    onClose();
  }
}
```

```
// App.jsx
<Alert
  type="information" heading="Success"
  closable
  onClose={() => console.log('closed')}
/>
```

BUENAS PRÁCTICAS

¿Por qué componentes de función?

También se pueden crear componentes con clases, pero en la comunidad de React predominan los componentes de función.



Menos código

Un componente es una función que devuelve JSX. Sin clases, ni constructores, ni this.



Lógica reutilizable

La lógica del componente se extrae y se comparte con más facilidad.



Hooks

useState y el resto de Hooks solo funcionan en componentes de función.

Panel Components: props y estado en vivo

Extensión de navegador para inspeccionar y depurar aplicaciones React.

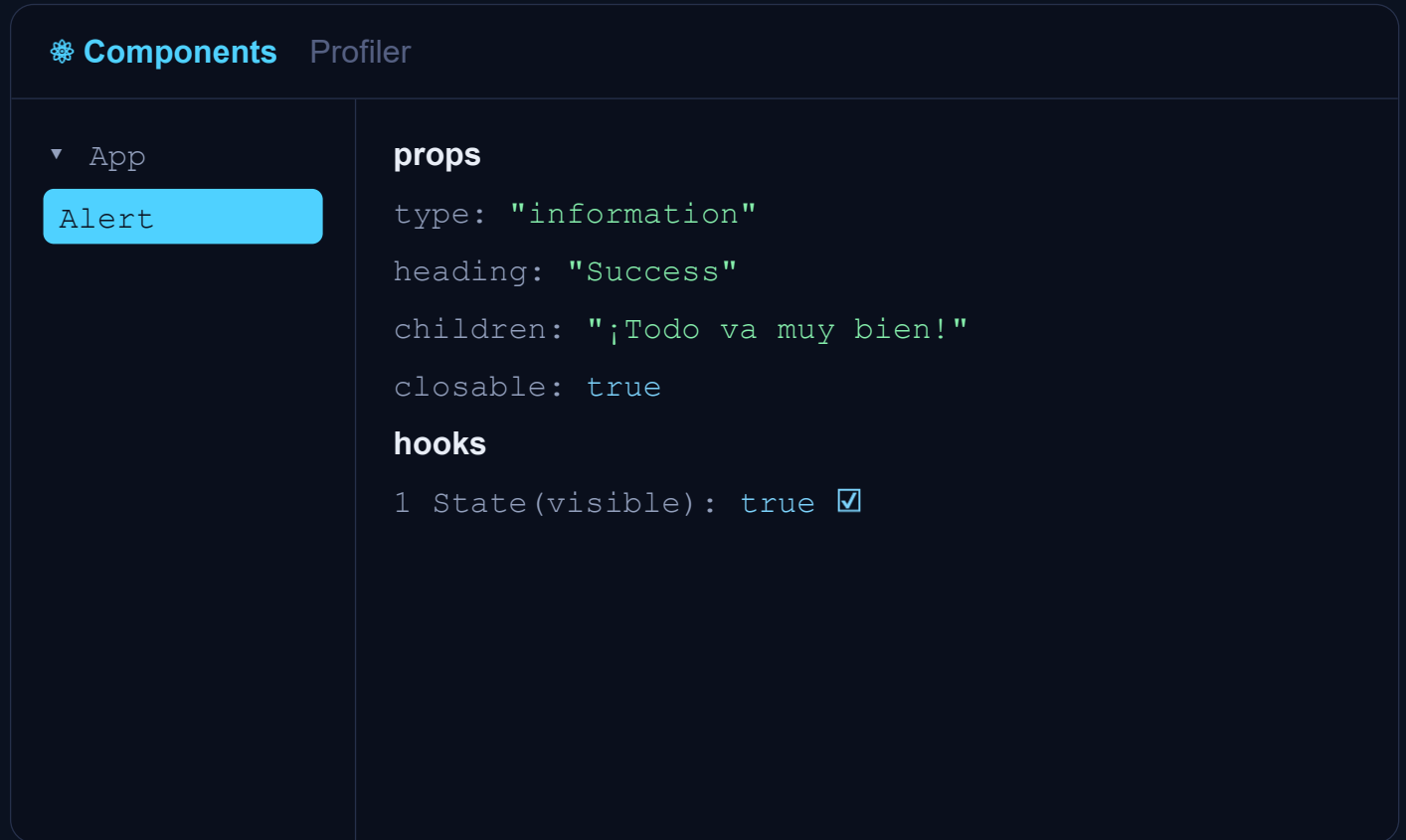
Chrome

Firefox

Edge

Se abre con **F12** en Windows o **Cmd + Opción + I** en Mac.

Puedes **editar props y estado** y ver el resultado al instante. El icono de varita muestra el nombre de las variables de estado.



Panel Profiler: medir los renderizados

01



Activa «Registrar por qué se renderizó cada componente».

02



Pulsa Iniciar perfilado (círculo azul).

03



Haz clic en el botón de cierre de la alerta.

04



Pulsa Detener perfilado (círculo rojo).

Línea de tiempo del perfil

App

Alert (0,7 ms)

Sirve para localizar rápidamente los componentes lentos en una interacción del usuario.

RESUMEN

Alert ya es configurable e interactivo



Props

Configuran el componente desde el JSX que lo usa.



Estado

useState guarda datos que cambian y provoca un re-render.



Eventos

Manejadores en JSX y props de función como onClose.



DevTools

Inspeccionan props, estado y rendimiento.

Próximos pasos

TypeScript en React

Tipar props

Tipar estado y eventos